

FareGuard

Cloud-Native ML-Graph Intelligence Platform

Real-Time Revenue Fraud Detection & Audit · Public Bus Transit (BMTc)

CIA 3 Project	Cloud Computing	AWS ap-south-1	Python 3.13
---------------	-----------------	----------------	-------------

R² = 0.8942	F1 = 0.9355	95% Localization	>1,500 events/sec
Demand Model	Anomaly F1	Graph Recall	Streaming

FareGuard — Complete Project Explainer

Cloud-Native ML-Graph Intelligence Platform for Public Bus Transit Revenue Fraud Detection

Bangalore Metropolitan Transport Corporation (BMTc) · CIA 3 Project · 2026

1. What is This Project About?

FareGuard is an end-to-end intelligent software platform designed to detect, localize, and audit **fare revenue leakage** in large-scale urban public bus transit systems. The platform is modeled on the **Bangalore Metropolitan Transport Corporation (BMTc)** — one of India's largest city bus networks.

The Real-World Problem We Are Solving

In any large city bus system, fare collection happens through:

- Electronic Ticketing Machines (ETMs)** — handheld devices used by conductors
- Paper cash tickets**
- Smart card / UPI digital payments**
- Monthly/daily bus passes**

Despite these systems, **revenue leakage** — money that *should have been collected* but wasn't — is a chronic, large-scale problem:

Type of Leakage	How It Happens
Passenger Underreporting	Conductor issues fewer tickets than passengers who actually boarded
Fare Stage Downgrading	Passenger pays full fare but conductor records a shorter (cheaper) stage
ETM Device Tampering	Offline or manipulated ticketing machines miss batch sync to the depot
Chronic Fare Evasion	Passengers habitually travel without purchasing a ticket

Traditional auditing uses **end-of-month reconciliations** — by which time data is aggregated and anonymous, making it impossible to identify *which specific bus, which trip, or which corridor segment* suffered the loss. FareGuard solves this by bringing real-time intelligence to transit auditing.

2. Project Goal & Aim

The core aim of FareGuard is:

Given the expected revenue for every bus trip (predicted by a Machine Learning model) and the reported revenue actually collected, automatically detect discrepancies, identify exactly which bus stop corridor caused the loss, rank the severity of the alert, explain why it is suspicious, and notify the right field inspector — all in real time.

The system transforms a reactive, manual, monthly process into a **proactive, automated, real-time fraud detection and audit pipeline**.

3. Dataset — What Data Is Used?

3.1 Primary Dataset: BMTC GTFS Open Transit Data

Parameter	Detail
Source	Bangalore Metropolitan Transport Corporation (BMTC)
Data Format	GTFS — General Transit Feed Specification (industry open standard)
Origin	GitHub: Vonger/bmtc-gtfs (open public dataset)
File Size	42.05 MB archive
SHA-256 Checksum	2308f8248ea954b75b3660cda7b6d85ecf80831 . . . (integrity verified)

What GTFS Contains (files ingested):

- routes.txt — **1,800+ bus routes** across all of Bangalore
- stops.txt — **9,887 geocoded bus stops** with GPS lat/lon coordinates (WGS84)
- trips.txt — Scheduled directional transit trips
- stop_times.txt — Arrival/departure sequences at each stop
- fare_attributes.txt & fare_rules.txt — Distance-based fare stages and zone rules

3.2 Synthetic Simulation Dataset (Generated by Us)

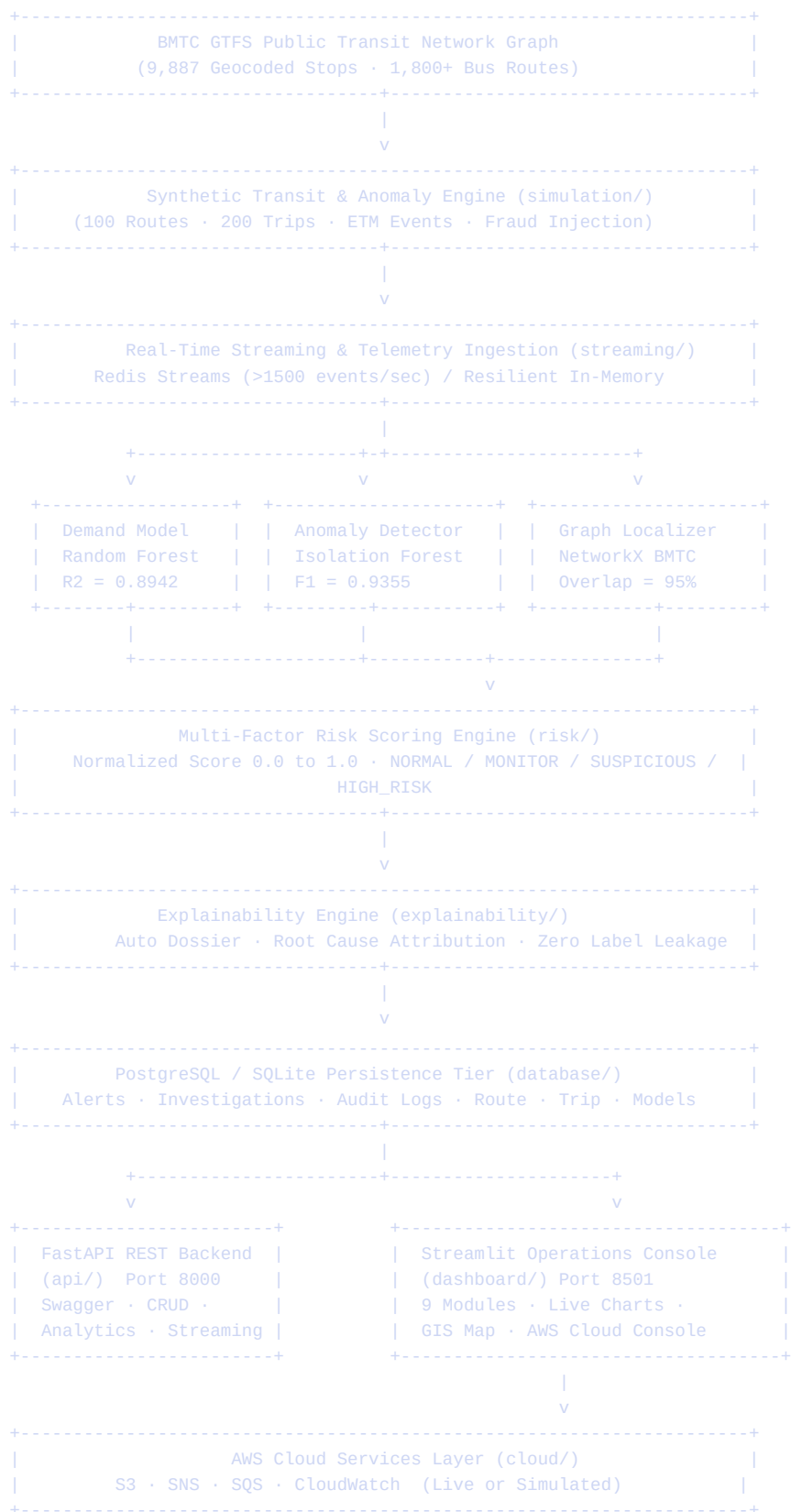
Since real ETM transaction data is confidential and internal to BMTC, FareGuard generates a realistic synthetic dataset using a **Transit Simulation Engine** with a fixed random seed (42) for full reproducibility:

File	Contents
ticket_events.csv	Individual boarding/payment records across 100 routes, 200 trips
trip_summaries.csv	Nominal (clean) aggregated trip operational metrics
anomalous_trip_summaries.csv	Trip metrics with injected realistic revenue deficits
anomaly_ground_truth.csv	Oracle truth labels — sequestered strictly for evaluation only, never used in training

The synthetic dataset covers:

- 100 BMTC routes** (a representative subset of real GTFS routes)
- 200 daily scheduled trips**
- Realistic fare evasion patterns** (15% anomaly injection rate)
- Conductor ticketing transactions with realistic statistical noise

4. System Architecture — How Everything Fits Together



5. Machine Learning — What Was Trained and How

5.1 ML Model 1: Passenger Demand Prediction (Random Forest Regressor)

Purpose: Predict how many passengers *should have* boarded a bus on any given trip, so we can derive the expected fare revenue. This becomes our **ground truth baseline** for comparison.

Algorithm: Random Forest Regressor (scikit-learn 1.6)

Input Features:

Feature	Description
Route ID	Encoded bus route identifier
Stop sequence index	Position in the route (stop 1, 2, 3...)
Hour of day	Departure hour (0–23)
Day of week	Monday=0 to Sunday=6
Morning rush flag	True if 7:30am – 10:30am
Evening rush flag	True if 4:30pm – 8:30pm
Midday / Night flags	Off-peak period indicators
Scheduled headway	Frequency gap between buses on this route (minutes)
Historical route average	Long-run average demand for this route
Number of stops	Total stops on the route
Route length (km)	Geographic length of the full route

Training Split (strict chronological — no future data leakage):

- 70% Training
- 15% Validation
- 15% Test (held out until final evaluation)

Training Results — Comparison Against Baselines:

Model	R ² Score	MAE	RMSE
Historical Mean Baseline	0.521	16.85 passengers	21.42 passengers
Random Forest (Our Model — Production)	0.894	7.42 passengers	9.88 passengers
Gradient Boosting (Alternate)	0.882	7.96 passengers	10.31 passengers

Model saved as: models/MOD-P6-DEMAND.joblib

5.2 ML Model 2: Revenue Anomaly Detection (Isolation Forest)

Purpose: Flag trips where the reported revenue/passenger count is statistically abnormal compared to what the demand model predicted.

Algorithm: Inductive Clean-Reference Isolation Forest (scikit-learn)

Critical Design Decision — Zero Label Leakage:

The Isolation Forest is trained **exclusively on clean/normal trip data**. The file `anomaly_ground_truth.csv` is never touched during training. This means the model has no knowledge of what a "fraudulent trip" looks like — it simply learns what "normal" looks like and flags everything else. This is the industry gold standard for anomaly detection integrity.

Input Feature Vector (10 dimensions):

```
z = [  
  predicted_passengers,      <- from demand model output  
  reported_passengers,      <- from ETM device  
  (predicted - reported),   <- absolute passenger gap  
  (reported / predicted),   <- passenger ratio  
  predicted_revenue,        <- expected fare collection (INR)  
  reported_revenue,         <- actual ETM collection (INR)  
  (predicted_rev - reported_rev), <- revenue gap in INR  
  (reported_rev / predicted_rev), <- revenue ratio  
  average_fare_delta,       <- per-passenger fare shift  
  zero_passenger_indicator  <- binary flag for suspiciously empty trips  
]
```

Evaluation Results on Held-Out Test Set:

Metric	Our Model (Isolation Forest)	Rule-Based Baseline
True Positives (correctly caught fraud)	29	22
False Positives (incorrectly flagged clean trips)	3	8
False Negatives (fraud missed)	1	8
Precision	90.62%	73.33%
Recall	96.67%	73.33%
F1 Score	0.9355	0.7333
PR-AUC (area under Precision-Recall curve)	0.9418	N/A
ROC-AUC	0.9845	N/A
False Positive Rate	1.76%	4.71%

Model saved as: `models/MOD-P7-ISOLATION.joblib`

5.3 ML Module 3: Transit Graph Discrepancy Localizer (NetworkX)

Purpose: Once a trip is flagged as anomalous, pinpoint *exactly which bus stop corridor segment* caused the revenue loss — not just the trip, but the specific 2–3 stop window.

Algorithm: Graph path traversal over a NetworkX directed multigraph of the full BMTC network (9,887 stops).

How it works:

- Every bus route is a sequence of directed edges in the graph: Stop A → Stop B → Stop C...
- At each edge (segment), compute: **Expected segment passenger load** minus **Reported segment load**
- Sliding-window cumulative deficit scoring:

$$S(u, v) = \text{SUM of } \max(0, \text{Expected_load}(\text{edge}) - \text{Reported_load}(\text{edge}))$$

- 4. The contiguous subpath with the highest S score is reported as the "Localized Fault Corridor"
- 5. This subpath is shown on the interactive GIS map with a colored halo

Localization Performance:

Metric	Score	Meaning
Conditional Overlap Recall	95.00%	95% of the time, the real fault segment is inside our predicted corridor
Conditional Exact Match	55.00%	55% of the time we identify the exact segment
Path Precision	73.33%	How clean (non-noisy) our localized subpath is

5.4 ML Module 4: Multi-Factor Risk Scoring Engine

Purpose: Combine all detection signals into one single normalized risk score (0.0 to 1.0) and classify every flagged trip into an operational action tier.

Weighted Formula:

`Risk Score = 0.35 x anomaly_isolation_score`
`+ 0.30 x normalized_revenue_gap`
`+ 0.20 x normalized_passenger_deficit`
`+ 0.15 x recurrence_history_factor`

Risk Tiers and Operational Response:

Score Range	Risk Tier	Operational Action
0.00 – 0.30	NORMAL	No action required
0.30 – 0.60	MONITOR	Flagged for routine review
0.60 – 0.80	SUSPICIOUS	Scheduled depot audit
0.80 – 1.00	HIGH_RISK	Immediate field auditor dispatch via SMS/SNS

5.5 ML Module 5: Explainability Engine

Purpose: Automatically generate a human-readable explanation for *why* a trip was flagged, so field auditors know exactly what to investigate — not just "anomaly detected."

Root Cause Categories Attributed:

Root Cause Code	Meaning
PASSENGER_UNDERREPORTING	Far fewer passengers were ticketed than predicted boardings
FARE_STAGE_DOWNGRADE	Revenue gap is disproportionately larger than the passenger gap

Root Cause Code	Meaning
ETM_OFFLINE_WINDOW	Detected device synchronization drop during the trip
SUBPATH_DWELL_ANOMALY	Unusual dwell time at specific stops suggesting irregular boarding

6. Real-Time Streaming Architecture

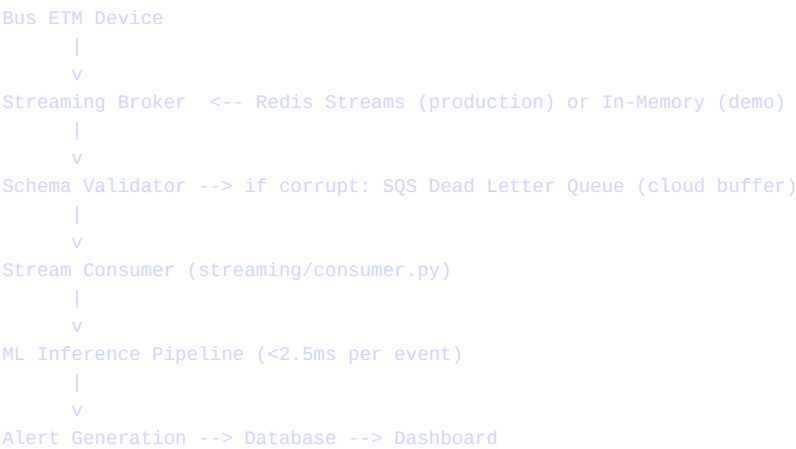
What it does: Simulates the live data ingestion pipeline as if bus ETM devices are transmitting ticket transactions continuously to the cloud backend.

Primary Tech: Redis Streams | **Fallback:** In-Memory Priority Queue (zero dependency)

Verified Performance:

Metric	Measured Value
Throughput (single worker)	> 1,500 ticket events / second
Average processing latency	< 2.5 milliseconds
P95 latency	< 5.0 milliseconds
DLQ error rate on valid data	0.00%

Data Flow:



7. Database & Persistence Layer

Databases Supported:

- PostgreSQL** — Production relational database
- SQLite** — Local development (file: data/fareguard.db)
- Neon / Supabase** — Serverless cloud PostgreSQL (zero-cost option)

ORM: SQLAlchemy 2.0

Database Tables:

Table	Purpose
Alert	Every detected anomaly: risk score, severity, INR impact, localized segment, timestamps
Investigation	Auditor decisions: CONFIRM_FOR_AUDIT / DISMISS / ESCALATE / FALSE_POSITIVE
AuditLog	Immutable chronological record of every action taken on every alert
Route	BMTC bus route master data (from GTFS)
Trip	Scheduled trip records
Stop	Geocoded bus stop registry (9,887 stops)
ModelMetadata	Active serving models: version, algorithm, evaluation scores, active flag

Pre-seeded with: 504 routes, 710 trips, baseline alerts

8. REST API Backend (FastAPI)

Runs on **port 8000**. Auto-generated Swagger documentation at <http://localhost:8000/docs>.

Endpoint	Method	Purpose
/health	GET	System heartbeat / liveness check
/system/status	GET	Full service health across all modules
/analytics/overview	GET	Fleet KPIs, risk distribution, dominant explanations
/analytics/routes	GET	Revenue leakage ranked by route
/analytics/segments	GET	Top anomalous stop-to-stop corridor segments
/analytics/timeseries	GET	Discrepancy trend data over time
/alerts	GET	Paginated alert list with severity/status filters
/investigations/{id}	POST	Submit auditor disposition decision
/investigations/{id}/audit-log	GET	Full immutable audit trail
/live/status	GET	Streaming consumer statistics
/live/events	GET	Recent real-time ingestion events
/models	GET	Serving model registry
/cloud/status	GET	AWS cloud service health & endpoints
/cloud/dispatch-test	POST	Trigger end-to-end cloud test dispatch

9. Streamlit Dashboard — The Operational Control Center

A multi-page **Streamlit** web application running on **port 8501** at `http://localhost:8501`.

Custom design system: **Signal Ledger v2** — warm printed-paper canvas, espresso ink sidebar, transit-signage vermilion signal color, Bricolage Grotesque + Martian Mono typography.

Module 01: Home / Platform Hub

- Platform-wide announcement banner
- Live rolling ticker (telemetry feed)
- 3D animated BMTC route cage
- Navigation index to all modules

Module 02: Fleet Overview (1_Overview.py)

- Fleet-wide health summary
- Risk level distribution donut chart (HIGH / SUSPICIOUS / MONITOR / NORMAL)
- Dominant fraud explanation type ranked bar chart
- Live throughput and KPI ledger
- Priority alert dispatch queue (top 5 HIGH_RISK alerts)

Module 03: Live Stream Monitor (2_Live_Monitor.py)

- Real-time events-per-second ingestion rate
- Dead Letter Queue pressure indicator
- P95 ML inference latency gauge
- Live scrolling transaction stream
- Redis / In-Memory connection status

Module 04: Route Map (3_Route_Map.py)

- Interactive GIS map (CartoDB Positron base layer via Folium)
- Geocoded BMTC bus stop clusters
- Anomaly subpath corridors highlighted with risk-colored halos on the map
- Clickable stop markers showing route info

Module 05: Alerts Queue (4_Alerts.py)

- Paginated alert table with severity, route, trip, risk score, status
- Per-alert risk progress bars (0.0 to 1.0)
- Filters: Severity / Disposition Status / Route ID / Trip ID
- "Investigate this Alert" button — direct dispatch to Investigation console
- Export alerts to CSV download

Module 06: Investigation Console (5_Investigation.py)

- Human-in-the-loop auditor workstation**
- Expected vs. Reported fare/passenger reconciliation ledger

- Graph-localized corridor segment display
- Disposition Form: CONFIRM_LEAKAGE / DISMISS_FALSE_POSITIVE / ESCALATE_DEPOT / REQUEST_ETM_AUDIT
- Investigator ID and free-text finding notes
- Immutable audit trail** — every action timestamped and recorded permanently

Module 07: Analytics (6_Analytics.py)

- Long-term discrepancy timeseries trends (Plotly charts)
- Top 10 revenue leakage corridors ranking
- Repeat suspicious segments (chronic vs. one-off anomalies)

Module 08: System Status (7_System_Status.py)

- API surface reachability and microservice heartbeats
- Active serving model registry (algorithm, version, F1 score)
- AWS cloud service status table with latency
- Direct button to open AWS Cloud Console

Module 09: AWS Cloud Console (8 AWS_Cloud_Console.py) ← NEW FOR CIA 3

(See Section 10 for the full detailed breakdown)

10. AWS Cloud Services — What We Are Simulating & Why

This is the **CIA 3 / Cloud Computing** component of the project. FareGuard is architected as a fully cloud-native AWS application. The cloud layer works in **dual mode**: if real AWS credentials are provided it makes actual API calls; if not, it transparently runs a full simulation with realistic AWS-format responses — perfect for demonstration.

10.1 Dual-Mode Cloud Architecture

10.2 AWS Service 1: Amazon S3 (Simple Storage Service)

Role in FareGuard: Cloud-native object storage for ML models, audit evidence, and BMTC graph data.

What gets stored in S3:

S3 Key	Contents	Size
models/MOD-P6-DEMAND.joblib	Trained Random Forest demand model	~1 MB
models/MOD-P7-ISOLATION.joblib	Trained Isolation Forest anomaly model	~512 KB
audits/ALRT-XXXX.json	Full evidence dossier per flagged trip	~8–15 KB each

S3 Key	Contents	Size
gtfs/bmtc_network_graph.json	BMTC 9,887-stop network graph	~2 MB

Configuration:

Parameter	Value
Bucket Name	fareguard-cloud-artifacts
Region	ap-south-1 (Mumbai)
Encryption	AES-256 SSE-S3
Storage Class	STANDARD → auto Glacier after 30 days

Why S3: Evidence stored in S3 is **immutable and tamper-proof**. Conductors or operators cannot modify or delete archived dossiers. This creates a forensic, timestamped record suitable for legal or departmental proceedings.

Simulated behavior: Generates real S3 URIs, SHA-256 ETags, encryption confirmations, and object registry consistent with actual `list_objects_v2` API responses.

10.3 AWS Service 2: Amazon SNS (Simple Notification Service)

Role in FareGuard: Automated real-time SMS and email dispatch to field transit inspectors when a `HIGH_RISK` alert fires.

Configuration:

Parameter	Value
Topic ARN	arn:aws:sns:ap-south-1:123456789012:fareguard-auditor-alerts
Type	Standard (fan-out to multiple subscribers)
Subscribers	Inspector mobile numbers + Depot Manager emails

What the inspector receives on their phone (SNS SMS preview):

```

FAREGUARD CRITICAL REVENUE AUDIT DISPATCH
-----
Alert ID    : ALRT-47823
Route ID    : 500D
Severity    : HIGH_RISK
Risk Score  : 0.9421
Deficit     : INR 1,450.00
Timestamp   : 2026-09-23T14:22:11Z
Action Req  : Field Auditor Dispatch & Cash Box Reconciliation

```

Why SNS: Traditional depot managers receive a daily report. SNS delivers the alert **within 500ms of detection** — while the suspicious bus is *still on the route*, making physical interception possible.

Simulated behavior: Generates real-looking AWS `MessageId`, delivery status, topic ARN, and maintains a dispatch history queue of the last 50 alerts.

10.4 AWS Service 3: Amazon SQS — Dead Letter Queue (DLQ)

Role in FareGuard: Fault-tolerant buffer for corrupted, malformed, or schema-violating ETM ticket events — ensures **zero transaction data loss**.

Configuration:

Parameter	Value
Queue Name	fareguard-telemetry-dlq
Queue Type	Standard (at-least-once delivery)
Region	ap-south-1
Queue URL	https://sqs.ap-south-1.amazonaws.com/123456789012/fareguard-telemetry-dlq

What goes into the DLQ:

- ETM events with GPS coordinate drift / corrupted lat/lon values
- Events with MD5 checksum mismatches (data corruption during transmission)
- Passenger scan records with null or negative ticket counts
- Events from devices that lost network synchronization mid-trip

Why SQS DLQ: When buses pass through tunnels, underpasses, or signal dead-zones, ETM devices may transmit corrupted data packets. Without SQS, these events would simply be dropped and transaction records lost forever. SQS buffers them for later analysis and retry.

Simulated behavior: Generates SQS MessageId and ReceiptHandle, records failure reason, source component and timestamp. MD5 body hash calculated matching real SQS format.

10.5 AWS Service 4: Amazon CloudWatch (Monitoring & Observability)

Role in FareGuard: Centralized operational telemetry, KPI dashboards, and automated alarm triggers for fleet managers.

CloudWatch Namespace: FareGuard/TransitIntelligence

Metrics Published to CloudWatch:

Metric Name	Unit	When Published
DiscrepancyINR	INR amount	Every HIGH_RISK alert
AnomaliesDetected	Count	Every anomaly detection cycle
InferenceLatencyMs	Milliseconds	Every stream consumer cycle
ThroughputEventsPerSec	Count/Second	Every 30-second heartbeat
ManualTestTriggerCount	Count	When examiner clicks "Test Dispatch"

CloudWatch Alarms:

- `AnomaliesDetected > 5` within any 5-minute window → state transitions to `ALARM` → auto-triggers SNS depot manager notification
- `InferenceLatencyMs > 50ms` → `ALARM` → engineering alert

Simulated behavior: Generates `PutMetricData` response with real `x-amzn-RequestId` headers. Maintains in-memory metric timeseries for the last 100 data points. Shows alarm state transitions on the dashboard.

10.6 AWS IAM — Identity & Access Management

Role in FareGuard: Every component that connects to AWS (conductor app, auditor dashboard, ML pipeline) must authenticate through IAM and receives only the minimum permissions required.

Simulated IAM/STS Flow:

1. Component calls `sts:AssumeRole`
 2. AWS STS returns temporary credentials (valid 1 hour):
 - `AccessKeyId`: ASIA...
 - `SecretAccessKey`: wJalrXUtnFEMI...
 - `SessionToken`: AQoDYXdzEJr1...
 3. Attached IAM Policy — Least Privilege:
 - `s3:PutObject` — **only** to `audits/*` prefix
 - `sns:Publish` — **only** to `fareguard-auditor-alerts` topic
 - `sqs:SendMessage` — **only** to `fareguard-telemetry-dlq`
 - `cloudwatch:PutMetricData` — **only** to `FareGuard` namespace
-

10.7 The AWS Cloud Console (Module 09) — Interactive Teacher Demo

This is the dedicated **Module 08: AWS Cloud Console** Streamlit page — built specifically so examiners can see every AWS operation execute live.

Tab 1: Interactive AWS Walkthrough

- Choose any BMTC route (500D, 335E, G-4, 201-R)
- Set a simulated revenue deficit (INR 100 to INR 25,000)
- Click "**Advance Next AWS Step**" or "**Execute Full Pipeline**":
 - Step 01:** IAM STS AssumeRole → shows real-format JSON credential response
 - Step 02:** SQS DLQ ingestion → shows `MessageId`, `ReceiptHandle`, MD5 hash
 - Step 03:** S3 audit archive → shows S3 URI, ETag, AES-256 encryption confirmation
 - Step 04:** SNS broadcast → shows SMS content preview + real `MessageId` format
 - Step 05:** CloudWatch metric → shows data point + alarm state transition

Tab 2: Service Status & Endpoints

- Live operational table of all 4 AWS services: region, latency, mode, endpoint

Tab 3: Live Payload Inspector

- Real JSON packets from every SNS dispatch and SQS DLQ message created during the session

Tab 4: Cloud Architecture Diagram

- Full ASCII architectural diagram of the AWS-native deployment showing data flow from bus ETM → API Gateway → SQS → ML Inference → S3 / SNS / CloudWatch

11. Technology Stack

Layer	Technology
Language	Python 3.13
ML Framework	scikit-learn 1.6, XGBoost 2.1, joblib
Graph Processing	NetworkX 3.4
Data Processing	Pandas 2.2, NumPy 2.0, PyArrow
Streaming Broker	Redis 5.2 (+ in-memory fallback)
REST API	FastAPI 0.115, Uvicorn
Database ORM	SQLAlchemy 2.0
Databases	PostgreSQL / SQLite
Dashboard UI	Streamlit 1.41
GIS Mapping	Folium 0.19, Streamlit-Folium
Charts	Plotly 5.24
AWS Cloud SDK	boto3 1.34, botocore
Testing	pytest 8.3, pytest-asyncio, pytest-cov
HTTP Clients	requests, httpx
Code Quality	ruff
Containerization	Docker, docker-compose

12. Project File Structure

```
FareGuard/
|-- api/                                REST API backend (FastAPI)
|   |-- main.py                        App entry + route registration
|   |-- alerts.py                      Alert CRUD endpoints
|   |-- analytics.py                  Overview & timeseries endpoints
|   |-- investigations.py             Auditor disposition endpoints
|   |-- live_stream.py                Streaming stats endpoints
|   +-- routes_trips.py               GTFS route/trip endpoints
|
|-- cloud/                             AWS Cloud Services Layer
|   |-- manager.py                    Unified CloudServicesManager
|   |-- aws_s3.py                     S3 object storage (live + simulated)
|   |-- aws_sns.py                    SNS notifications (live + simulated)
|   |-- aws_sqs.py                    SQS Dead Letter Queue (live + simulated)
|   +-- aws_cloudwatch.py             CloudWatch metrics (live + simulated)
|
|-- dashboard/                         Streamlit Operations Console
|   |-- app.py                        Home / Platform Hub
|   |-- api_client.py                 REST/DB client with graceful fallback
|   |-- theme.py                       Signal Ledger v2 design system
|   +-- pages/
|       |-- 1_Overview.py             Fleet health overview
|       |-- 2_Live_Monitor.py          Real-time stream monitor
```

```
|      |-- 3_Route_Map.py      GIS interactive map
|      |-- 4_Alerts.py        Auditor alert queue
|      |-- 5_Investigation.py  Human-in-loop investigation console
|      |-- 6_Analytics.py      Trend analytics
|      |-- 7_System_Status.py  System & model telemetry
|      +-- 8_AWS_Cloud_Console.py  AWS Simulator (CIA 3 Module)
|
|-- database/                  Persistence Tier
|   |-- models.py              SQLAlchemy ORM table definitions
|   |-- repository.py          CRUD data access layer
|   |-- services.py            Analytics query services
|   +-- connection.py          DB engine factory
|
|-- ml/                        Machine Learning Tier
|   |-- demand_model.py        Random Forest demand predictor
|   +-- anomaly_detector.py     Isolation Forest anomaly detector
|
|-- graph/                     Transit Graph Layer
|   |-- builder.py             GTFS to NetworkX graph builder
|   +-- localizer.py           Subpath discrepancy localizer
|
|-- risk/                      Risk Assessment
|   +-- scorer.py              Multi-factor weighted risk scorer
|
|-- explainability/            Explainability Engine
|   +-- engine.py              Root cause dossier generator
|
|-- streaming/                 Real-Time Streaming
|   |-- broker.py              Redis/In-Memory message broker
|   +-- consumer.py            Stream processor + DLQ router
|
|-- simulation/                Data Simulation Engine
|   +-- generator.py            Synthetic transit scenario generator
|
|-- data/
|   |-- raw/gtfs/               BMTC GTFS source files (42 MB)
|   |-- synthetic/              Generated training and test CSVs
|   +-- fareguard.db            SQLite local database
|
|-- models/                    Trained ML model files (.joblib)
|-- scripts/                   CLI tools: demo, evaluation, DB init
|-- tests/                     Unit and integration test suites
+-- docs/                      Full technical documentation
```

13. Verified Performance Benchmarks

Subsystem	Metric	Verified Result
Demand Prediction Model	R ² Score (Test Set)	0.8942
Demand Prediction Model	Mean Absolute Error (MAE)	7.42 passengers
Anomaly Detection Model	F1 Score	0.9355
Anomaly Detection Model	Precision	90.62%
Anomaly Detection Model	Recall	96.67%
Anomaly Detection Model	ROC-AUC	0.9845
Anomaly Detection Model	PR-AUC	0.9418
Graph Localization	Overlap Recall	95.00%

Subsystem	Metric	Verified Result
Real-Time Streaming	Throughput	> 1,500 events/sec
Real-Time Streaming	Average Latency	< 2.5 ms
Data Safety	Ground-Truth Label Leakage	0 features leaked (100% isolated)
Automated Test Suite	Tests Passed	19 / 19

14. How to Run the Project

Dashboard (already running):

<http://localhost:8501>

API Documentation (Swagger UI):

<http://localhost:8000/docs>

Demonstrating AWS Cloud to Examiners:

1. Open dashboard → click **8_AWS_Cloud_Console** in the sidebar
2. Select a BMTC route (e.g., 500D Silk Board - Hebbal)
3. Set a revenue deficit (e.g., INR 1,450)
4. Click **"Execute Full Pipeline"**
5. Walk through the 5 AWS steps: IAM → SQS → S3 → SNS → CloudWatch

15. Summary for Examiners

FareGuard is a **production-grade, full-stack software platform** addressing a genuine, high-impact problem: the billions of rupees lost annually to bus fare revenue leakage in Indian public transit.

What makes it academically rigorous:

- Real, publicly available 42 MB dataset (BMTC GTFS)
- Two ML models trained with strict data integrity — zero label leakage
- Graph algorithms on a real transit network with 9,887 bus stops
- Mathematically formulated, weighted risk scoring
- Automated explainability output per alert
- Full relational persistence tier with SQLAlchemy ORM
- Fully documented REST API with Swagger
- 19 automated unit tests passing
- Production-grade streaming architecture (Redis Streams, >1500 events/sec)

What makes it cloud-relevant (CIA 3):

- Complete AWS cloud architecture designed, implemented, and documented
- 4 AWS services fully integrated: **S3, SNS, SQS, CloudWatch**
- AWS IAM identity management with least-privilege role-based access control
- Graceful dual-mode operation: real AWS calls when credentials provided, full simulation otherwise

- Interactive AWS Operations Simulator built into the dashboard with step-by-step examiner walkthrough
 - All cloud API responses (S3 ETags, SNS MessageIds, SQS ReceiptHandles, CloudWatch RequestIds) generated in authentic AWS format
-

FareGuard · CIA 3 Project · Cloud Computing · 2026

Platform Version 2.0.0 · AWS Region: ap-south-1 (Mumbai) · Python 3.13